

3. Eventos

Los eventos son acciones desencadenadas por la interacción del usuario u otros actores. Estos eventos van a permitir que el programa interactúe con sus peticiones, por ejemplo, si hemos programado una calculadora, esta tendrá que ser capaz de reaccionar a las pulsaciones de los botones por parte del usuario para decidir qué número y operaciones debe usar.

El evento, dentro de nuestro programa, desencadenará la ejecución de una función o parte de nuestro código, y podrá ser generado por diferentes fuentes, como la pulsación de un botón por parte del usuario (pulsar un botón, pulsar una tecla, etc.).

¿Cómo se programan los eventos?

Para programar los eventos haremos uso de interfaces (como la interfaz de AWT [ActionListener](#)); estas interfaces proporcionan funcionalidad a nuestras clases para reaccionar a estos eventos. La interfaz ActionListener nos forzará a implementar el método actionPerformed que será invocado cuando ocurra la acción. Es en este método donde debemos poner el código que queremos ejecutar cuando se desencadene la acción.

¿Cómo indico a un componente de mi interfaz gráfica que genere eventos?

Para que un componente de nuestras interfaces gráficas genere eventos debemos usar su método addActionListener, que no devuelve nada y recibe como parámetro cualquier objeto que implemente la interfaz ActionListener (polimorfismo):

```
boton1.addActionListener(ActionListener l);
```

En este [enlace](#) puedes encontrar información sobre los diferentes listeners que pueden implementarse en cada componente.

Eventos de teclado

Como se mostraba en el enlace anterior, además de ActionListener, los elementos de nuestras interfaces gráficas pueden reaccionar a más eventos.

Para leer eventos del teclado usaremos la interfaz [KeyListener](#). Esta interfaz nos obliga a implementar tres métodos abstractos:

- `keyPressed(KeyEvent e)`: se invoca al pulsar una tecla.
- `keyReleased(KeyEvent e)`: se invocará al soltar la tecla.
- `keyTyped(KeyEvent e)`: se invocará cuando la tecla que hemos pulsado genere un carácter Unicode. Esto es útil cuando queremos detectar eventos relacionados con la escritura. No se activará, por ejemplo, con las teclas Alt y Control.

Para añadir un KeyListener en alguno de nuestros componentes usamos `addKeyListener`, pasando como parámetro la instancia de un objeto que implementa la interfaz KeyListener, de la siguiente forma:

```
componente.addKeyListener(KeyListener k);
```

Eventos del ratón

Para leer eventos del ratón usaremos la interfaz [MouseListener](#). Esta interfaz nos obliga a implementar cinco métodos abstractos:

- `mouseClicked(MouseEvent e)`: Al hacer click con el ratón (pulsar y soltar) sobre el componente.
- `mousePressed(MouseEvent e)`: Al pulsar el botón del ratón sobre el componente.
- `mouseReleased(MouseEvent e)`: Al soltar el botón del ratón sobre el componente.
- `mouseEntered(MouseEvent e)`: Cuando el puntero del ratón entra sobre el área del componente.
- `mouseExited(MouseEvent e)`: Cuando el puntero del ratón sale del área del componente.

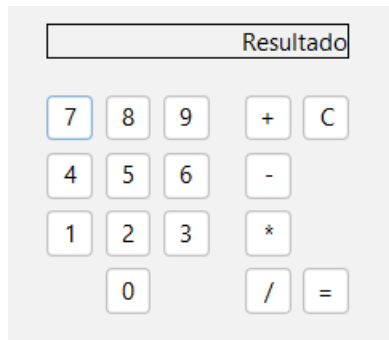
Para añadir un MouseListener en alguno de nuestros componentes usamos `addMouseListener`, pasando como parámetro la instancia de un objeto que implementa la interfaz MouseListener, de la siguiente forma:

```
componente.addMouseListener(MouseListener m);
```

Ejercicios

Ejercicio 1.

Crea una calculadora como la siguiente:



Donde todo son botones, y “Resultado” es un JLabel. El botón C sirve para eliminar toda la información almacenada.

Ejercicio 2.

Prueba la gestión de eventos con otros componentes, como JCheckBox, JRadioButton o JComboBox.

Ejercicio 3.

Experimenta con el JFrame4 del código proporcionado, para ver qué teclas se activan con el método keyTyped.

Ejercicio 4.

Usando el código proporcionado con la lección, emplea las herramientas de depuración sobre la clase “ReaccionEvento” para analizar la instancia “e” de “ActionEvent”. ¿Qué métodos o propiedades aporta? ¿Piensa algún uso de estas?

Ejercicio 5.

En la línea 32 del main del programa proporcionado con la lección tenemos esta línea:

```
boton2.addActionListener(new ActionListener(){
```

Se genera un warning sobre ella. ¿Qué significa? Intenta hacer la solución que el IDE te propone. ¿No lo tienes claro del todo? Explora los recursos de ampliación de la unidad.